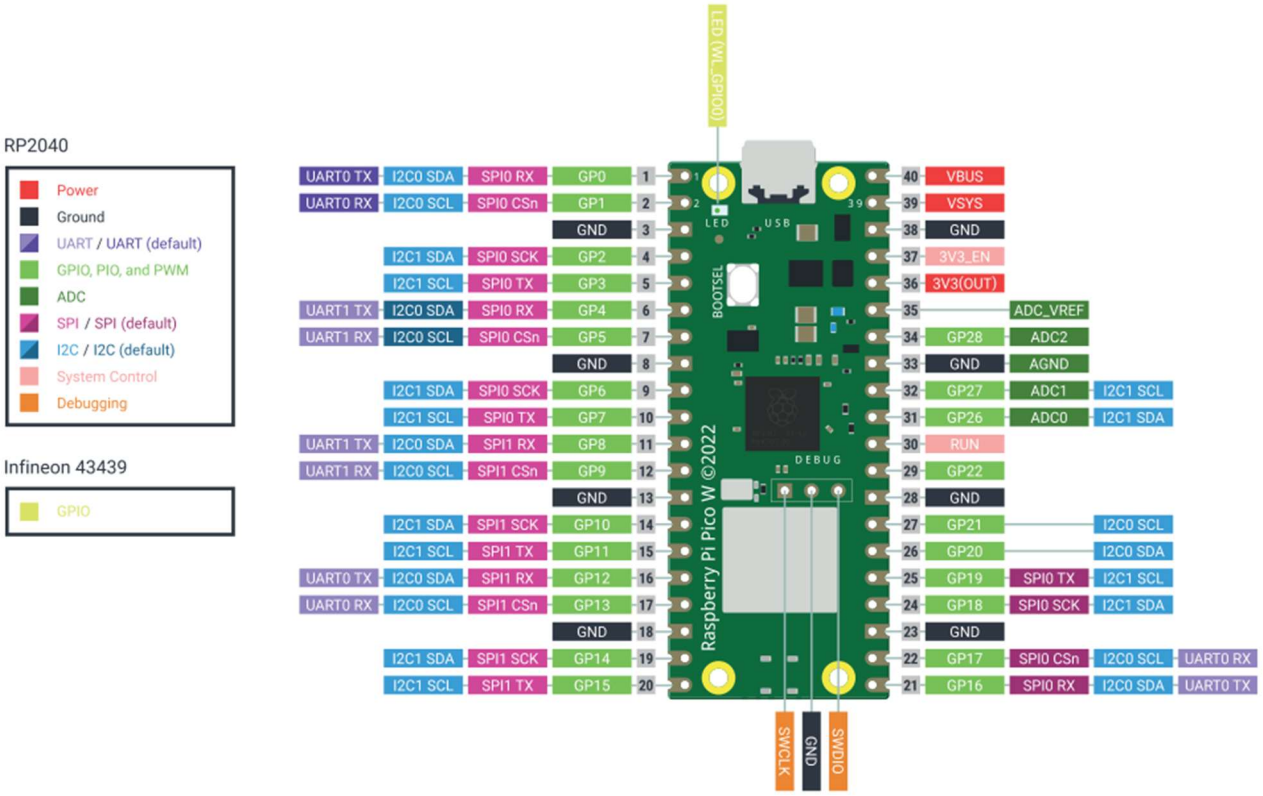
Hands-On Practice Assignments: Week 4



Experiment 15: Interface HCSR04 Ultrasonic sensor with Pico W and display the results

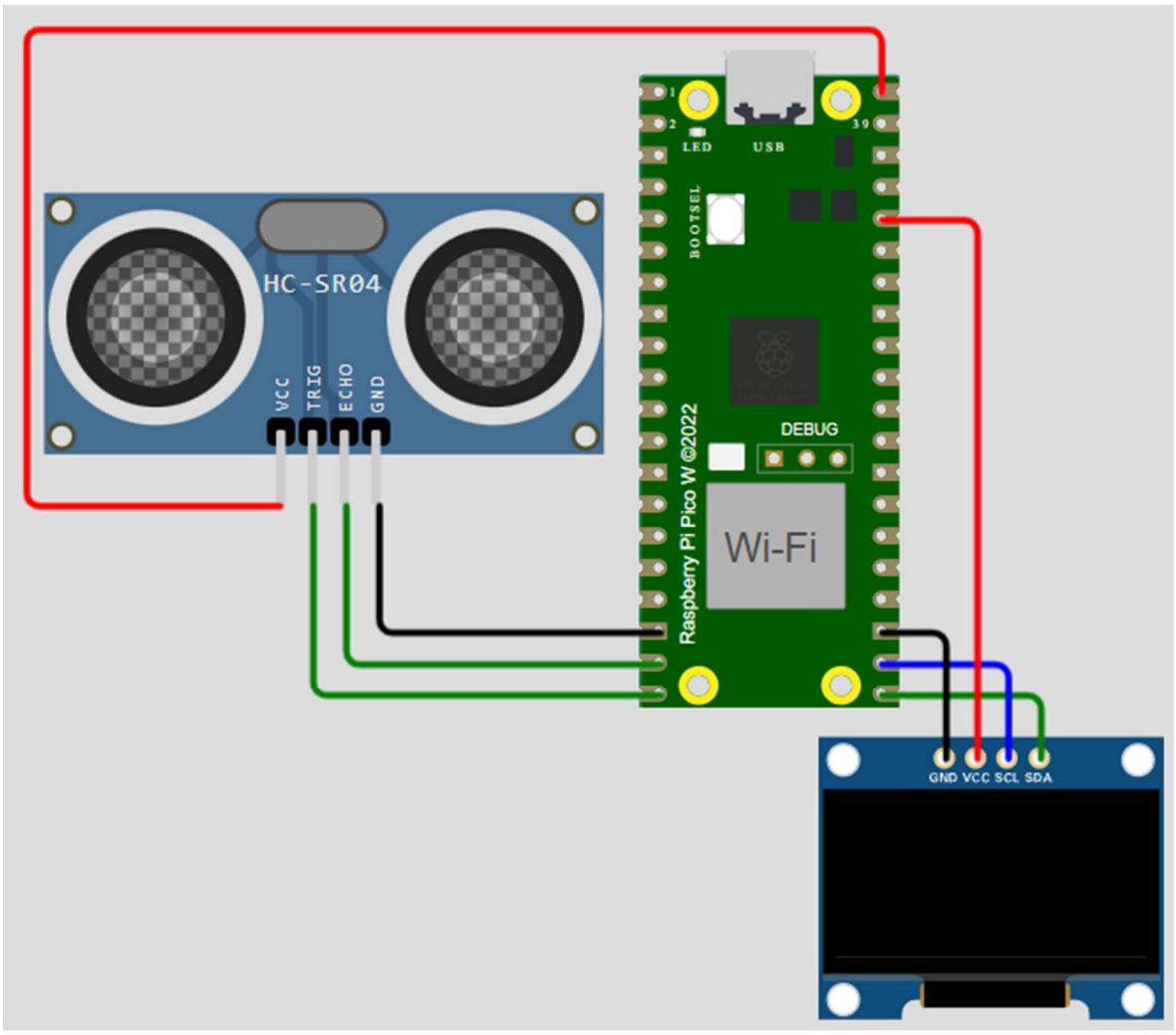


Fig 10: Circuit diagram for 15

#Code

#main.py

```
from machine import Pin, I2C
from ssd1306 import SSD1306_I2C
import utime

WIDTH = 128
HEIGHT = 64
i2c = I2C(0, scl = Pin(17), sda = Pin(16), freq=400000)
display = SSD1306_I2C(WIDTH, HEIGHT, i2c)

display.text('Welcome to',14,0)
display.text('AICTE IDEA LAB',0,12)
```

```
display.text('Interfacing',0,36)
display.text('UltrasonicSensor', 0,48)
display.show()
utime.sleep(10)
display.fill(0)

trigger = Pin(15, Pin.OUT)
echo = Pin(14, Pin.IN)

def distCalc():
    trigger.low()
    utime.sleep_us(2)
    trigger.high()
    utime.sleep_us(5)
    trigger.low()
    while echo.value() == 0:
        signaloff = utime.ticks_us()
    while echo.value() == 1:
        signalon = utime.ticks_us()
    timePassed = signalon - signaloff
    distance = (timePassed * 0.0343) / 2
    print("The distance from object is ", distance, "cm")
    return distance

while True:
    dist_cm = distCalc()
    display.text('Dist. of Object', 0,0)
    display.text("{:.2f} cm".format(dist_cm), 0, 48)
    display.show()
    display.fill(0)
```

#ssd1306.py

```
# MicroPython SSD1306 OLED driver, I2C and SPI interfaces

from micropython import const
import framebuf

# register definitions
SET_CONTRAST = const(0x81)
SET_ENTIRE_ON = const(0xA4)
SET_NORM_INV = const(0xA6)
SET_DISP = const(0xAE)
SET_MEM_ADDR = const(0x20)
SET_COL_ADDR = const(0x21)
SET_PAGE_ADDR = const(0x22)
SET_DISP_START_LINE = const(0x40)
SET_SEG_REMAP = const(0xA0)
SET_MUX_RATIO = const(0xA8)
```

```
SET_IREF_SELECT = const(0xAD)
SET_COM_OUT_DIR = const(0xC0)
SET_DISP_OFFSET = const(0xD3)
SET_COM_PIN_CFG = const(0xDA)
SET_DISP_CLK_DIV = const(0xD5)
SET_PRECHARGE = const(0xD9)
SET_VCOM_DESEL = const(0xDB)
SET_CHARGE_PUMP = const(0x8D)

# Subclassing FrameBuffer provides support for graphics primitives
# http://docs.micropython.org/en/latest/pyboard/library/framebuf.html
class SSD1306(framebuf.FrameBuffer):
    def __init__(self, width, height, external_vcc):
        self.width = width
        self.height = height
        self.external_vcc = external_vcc
        self.pages = self.height // 8
        self.buffer = bytearray(self.pages * self.width)
        super().__init__(self.buffer, self.width, self.height, framebuf.MONO_VLSB)
        self.init_display()

    def init_display(self):
        for cmd in (
            SET_DISP,  # display off
            # address setting
            SET_MEM_ADDR,
            0x00,  # horizontal
            # resolution and layout
            SET_DISP_START_LINE,  # start at line 0
            SET_SEG_REMAP | 0x01,  # column addr 127 mapped to SEG0
            SET_MUX_RATIO,
            self.height - 1,
            SET_COM_OUT_DIR | 0x08,  # scan from COM[N] to COM0
            SET_DISP_OFFSET,
            0x00,
            SET_COM_PIN_CFG,
            0x02 if self.width > 2 * self.height else 0x12,
            # timing and driving scheme
            SET_DISP_CLK_DIV,
            0x80,
            SET_PRECHARGE,
            0x22 if self.external_vcc else 0xF1,
            SET_VCOM_DESEL,
            0x30,  # 0.83*Vcc
            # display
            SET_CONTRAST,
            0xFF,  # maximum
            SET_ENTIRE_ON,  # output follows RAM contents
```

```
            SET_NORM_INV,  # not inverted
            SET_IREF_SELECT,
            0x30,  # enable internal IREF during display on
            # charge pump
            SET_CHARGE_PUMP,
            0x10 if self.external_vcc else 0x14,
            SET_DISP | 0x01,  # display on
        ):  # on
            self.write_cmd(cmd)
        self.fill(0)
        self.show()

    def poweroff(self):
        self.write_cmd(SET_DISP)

    def poweron(self):
        self.write_cmd(SET_DISP | 0x01)

    def contrast(self, contrast):
        self.write_cmd(SET_CONTRAST)
        self.write_cmd(contrast)

    def invert(self, invert):
        self.write_cmd(SET_NORM_INV | (invert & 1))

    def rotate(self, rotate):
        self.write_cmd(SET_COM_OUT_DIR | ((rotate & 1) << 3))
        self.write_cmd(SET_SEG_REMAP | (rotate & 1))

    def show(self):
        x0 = 0
        x1 = self.width - 1
        if self.width != 128:
            # narrow displays use centred columns
            col_offset = (128 - self.width) // 2
            x0 += col_offset
            x1 += col_offset
        self.write_cmd(SET_COL_ADDR)
        self.write_cmd(x0)
        self.write_cmd(x1)
        self.write_cmd(SET_PAGE_ADDR)
        self.write_cmd(0)
        self.write_cmd(self.pages - 1)
        self.write_data(self.buffer)


class SSD1306_I2C(SSD1306):
    def __init__(self, width, height, i2c, addr=0x3C, external_vcc=False):
        self.i2c = i2c
```

```
        self.addr = addr
        self.temp = bytearray(2)
        self.write_list = [b"\x40", None]  # Co=0, D/C#=1
        super().__init__(width, height, external_vcc)

    def write_cmd(self, cmd):
        self.temp[0] = 0x80  # Co=1, D/C#=0
        self.temp[1] = cmd
        self.i2c.writeto(self.addr, self.temp)

    def write_data(self, buf):
        self.write_list[1] = buf
        self.i2c.writevto(self.addr, self.write_list)


class SSD1306_SPI(SSD1306):
    def __init__(self, width, height, spi, dc, res, cs, external_vcc=False):
        self.rate = 10 * 1024 * 1024
        dc.init(dc.OUT, value=0)
        res.init(res.OUT, value=0)
        cs.init(cs.OUT, value=1)
        self.spi = spi
        self.dc = dc
        self.res = res
        self.cs = cs
        import time

        self.res(1)
        time.sleep_ms(1)
        self.res(0)
        time.sleep_ms(10)
        self.res(1)
        super().__init__(width, height, external_vcc)

    def write_cmd(self, cmd):
        self.spi.init(baudrate=self.rate, polarity=0, phase=0)
        self.cs(1)
        self.dc(0)
        self.cs(0)
        self.spi.write(bytearray([cmd]))
        self.cs(1)

    def write_data(self, buf):
        self.spi.init(baudrate=self.rate, polarity=0, phase=0)
        self.cs(1)
        self.dc(1)
        self.cs(0)
        self.spi.write(buf)
        self.cs(1)
```

Experiment 16: Interface HCSR501 PIR sensor with Pico W and display the results

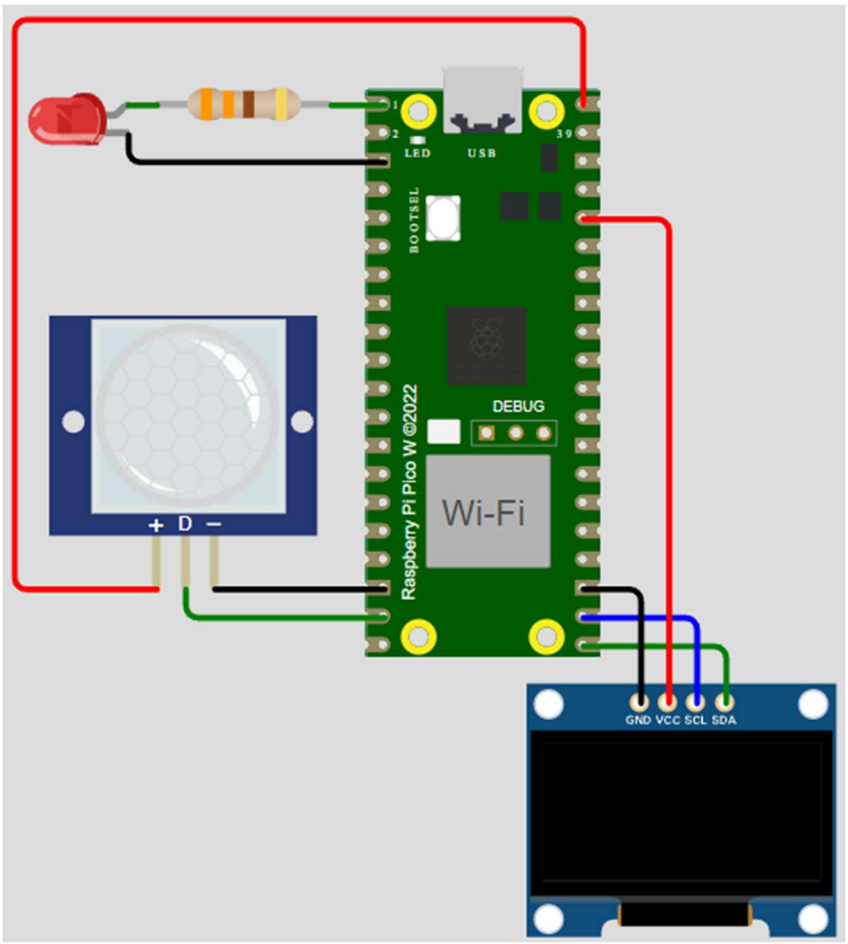


Fig 11: Circuit diagram for 16

Morse code is a method used in telecommunication to encode text characters as standardized sequences of two different signal durations, called dots and dashes. It is used by tapping the combination of dots and dashes needed and pausing for the correct gap duration. There are longer gaps between words than letters in a word.

So on detection of motion, we shall flash LED in MORSE code for SOS (the letter "s" is three dots and "o" is three dashes).

#Code

#main.py

```
from machine import Pin, I2C
from ssd1306 import SSD1306_I2C
import utime

WIDTH = 128
HEIGHT = 64
```

```
i2c = I2C(0, scl = Pin(17), sda = Pin(16), freq=400000)
display = SSD1306_I2C(WIDTH, HEIGHT, i2c)

display.text('Welcome to',14,0)
display.text('AICTE IDEA LAB',0,12)
display.text('Interfacing',0,36)
display.text('PIR Sensor', 0,48)
display.show()
utime.sleep(2)
display.fill(0)

pir = Pin(14, Pin.IN, Pin.PULL_DOWN)
led= Pin(0, Pin.OUT)

def morseSOS():
    dot = 0.25
    dash = 0.5
    gap = 0.2
    #Morse code for S: ...
    for i in range(0,3):
        led.on()
        utime.sleep(dot)
        led.off()
        utime.sleep(gap)
    utime.sleep(0.75)
    #Morse code for ): ---
    for i in range(0,3):
        led.on()
        utime.sleep(dash)
        led.off()
        utime.sleep(gap)
    utime.sleep(0.75)
    #Morse code for S: ...
    for i in range(0,3):
        led.on()
        utime.sleep(dot)
        led.off()
        utime.sleep(gap)
    utime.sleep(1.5)

while True:
    display.fill(0)
    if pir.value() == 1:
        display.text('MOTION', 0,0)
        display.text('DETECTED', 0, 24)
        display.show()
        morseSOS()
        utime.sleep(3)
```

```
        display.fill(0)
        led.off()
    else:
        led.off()
        display.text('NO EVENT', 0,0)
        display.show()
        utime.sleep(0.5)
        display.fill(0)
    utime.sleep(1)
```

#ssd1306.py Refer to code on pg 61.

Experiment 17: Interface MPU6050 Accelerometer and gyroscope with Pico W and display the results.

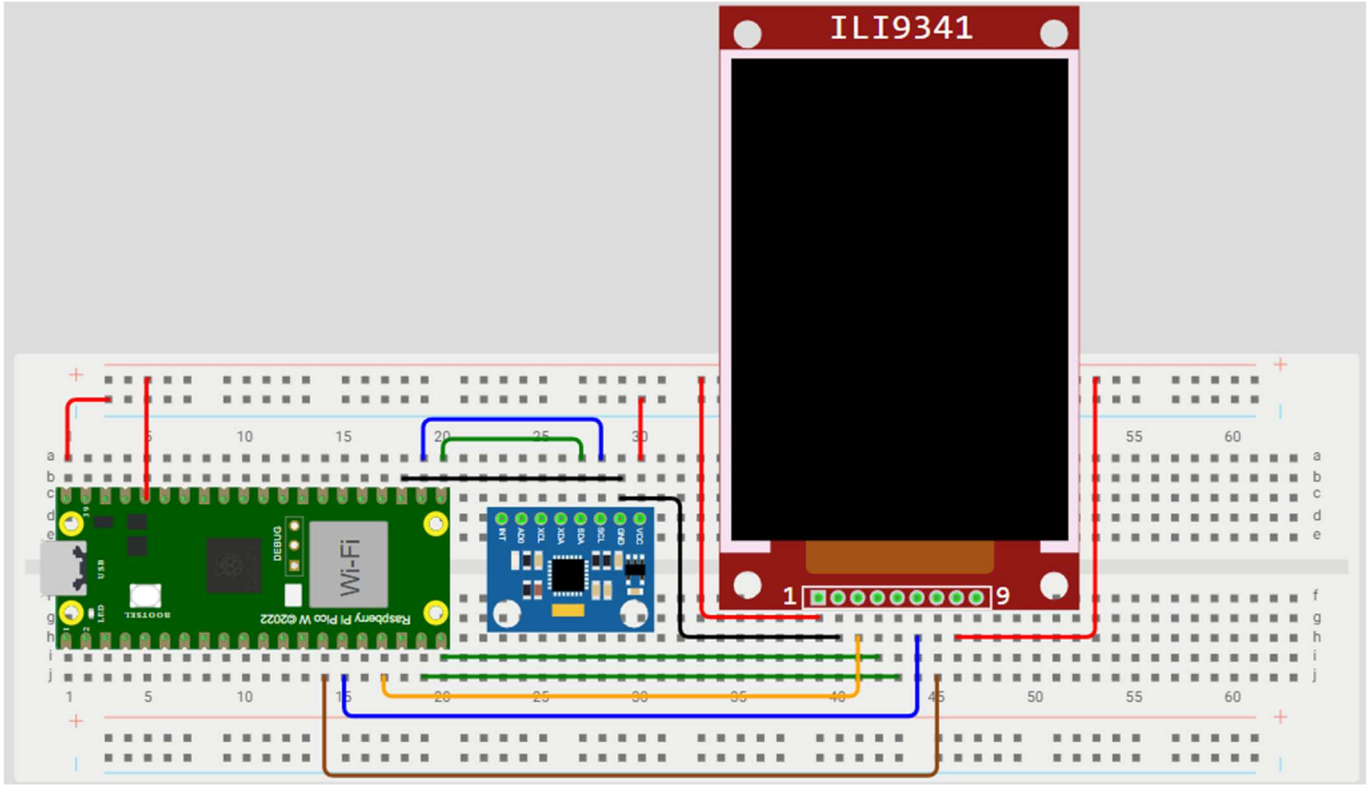


Fig 12: Circuit diagram for 17

#Code

#main.py

```
from machine import Pin, SPI, I2C
import utime
from ili9341 import ILI9341, color565  # Make sure ili9341.py is in the same folder
from mpu6050 import init_mpu6050, get_mpu6050_data
```

```
import math

# === SPI Configuration for Pico W ===
spi = SPI(1, baudrate=40000000, polarity=0, phase=0,
          sck=Pin(10), mosi=Pin(11), miso=None)
# === Control Pins ===
cs  = Pin(13, Pin.OUT)
dc  = Pin(14, Pin.OUT)
rst = Pin(15, Pin.OUT)
WIDTH = 240
HEIGHT = 320
# === Initialize SPI for tft ===
tft = ILI9341(spi, cs=cs, dc=dc, rst=rst, WIDTH, HEIGHT)
tft.fill(color565(0, 0, 0))  # Clear screen black

#Initial Text
tft.text('WELCOME TO AICTE IDEA LAB',0,0,color565(255, 255, 0))
tft.text('Interfacing MPU6050',0,10,color565(255, 255, 0))

#== I2C configuration
i2c = I2C(0, scl = Pin(17), sda = Pin(16), freq=400000)
init_mpu6050(i2c) #Initialize MPU6050

def calculate_tilt_angles(accel_data):
    #Calculating tilt angles from accelerometer data
    x, y, z = accel_data['x'], accel_data['y'], accel_data['z']

    tilt_x = math.atan2(y, math.sqrt(x * x + z * z)) * 180 / math.pi
    tilt_y = math.atan2(-x, math.sqrt(y * y + z * z)) * 180 / math.pi
    tilt_z = math.atan2(z, math.sqrt(x * x + y * y)) * 180 / math.pi

    return tilt_x, tilt_y, tilt_z

def complementary_filter(pitch, roll, gyro_data, dt, alpha=0.98):
    # Calculating pitch and roll from gyroscope data
    pitch += gyro_data['x'] * dt
    roll -= gyro_data['y'] * dt

    pitch = alpha * pitch + (1 - alpha) * math.atan2(gyro_data['y'],
math.sqrt(gyro_data['x'] * gyro_data['x'] + gyro_data['z'] * gyro_data['z'])) * 180 /
math.pi
    roll = alpha * roll + (1 - alpha) * math.atan2(-gyro_data['x'],
math.sqrt(gyro_data['y'] * gyro_data['y'] + gyro_data['z'] * gyro_data['z'])) * 180 /
math.pi

    return pitch, roll

pitch = 0
```

```
roll = 0
prev_time = utime.ticks_ms()

while True:
    tft.fill(color565(0, 0, 0))  # Clear

    data = get_mpu6050_data(i2c)
    curr_time = utime.ticks_ms()
    dt = (curr_time - prev_time) / 1000

    tilt_x, tilt_y, tilt_z = calculate_tilt_angles(data['accel'])
    pitch, roll = complementary_filter(pitch, roll, data['gyro'], dt)

    prev_time = curr_time

    tft.text("Temperature: {:.2f} degC".format(data['temp']),0,20, color565(0,255,255))
    tft.text("Accln (X,Y,Z) g",0,30,color565(0,255,255))
    tft.text("{:.2f}, {:.2f}, {:.2f}".format(data['accel']['x'], data['accel']['y'],
data['accel']['z']),0,40,color565(255,255,255))
    tft.text("Gyro: (X,Y,Z) deg/s",0,50,color565(0,255,255))
    tft.text("{:.2f}, {:.2f}, {:.2f} deg/s".format(data['gyro']['x'], data['gyro']['y'],
data['gyro']['z']),0,60,color565(255,255,255))
    tft.text("Tilt(deg): (X,Y,Z)",0,70, color565(0,255,255))
    tft.text("{:.2f}, {:.2f}, {:.2f}".format(tilt_x, tilt_y,
tilt_z),0,80,color565(255,255,255))
    tft.text("Pitch(deg: {:.2f}".format(pitch),0,90, color565(255, 0, 255))
    tft.text("Roll(deg): {:.2f}".format(roll),0,100,color565(255, 0, 255))
    utime.sleep(1)
```

#mpu6050.py

```
from machine import Pin, I2C
import utime

PWR_MGMT_1 = 0x6B
SMPLRT_DIV = 0x19
CONFIG = 0x1A
GYRO_CONFIG = 0x1B
ACCEL_CONFIG = 0x1C
TEMP_OUT_H = 0x41
ACCEL_XOUT_H = 0x3B
GYRO_XOUT_H = 0x43

def init_mpu6050(i2c, address=0x68):
    i2c.writeto_mem(address, PWR_MGMT_1, b'x00')
    utime.sleep_ms(100)
    i2c.writeto_mem(address, SMPLRT_DIV, b'x07')
    i2c.writeto_mem(address, CONFIG, b'x00')
```

```
    i2c.writeto_mem(address, GYRO_CONFIG, b'x00')
    i2c.writeto_mem(address, ACCEL_CONFIG, b'x00')

def read_raw_data(i2c, addr, address=0x68):
    high = i2c.readfrom_mem(address, addr, 1)[0]
    low = i2c.readfrom_mem(address, addr + 1, 1)[0]
    value = high << 8 | low
    if value > 32768:
        value = value - 65536
    return value

def get_mpu6050_data(i2c):
    temp = read_raw_data(i2c, TEMP_OUT_H) / 340.0 + 36.53
    accel_x = read_raw_data(i2c, ACCEL_XOUT_H) / 16384.0
    accel_y = read_raw_data(i2c, ACCEL_XOUT_H + 2) / 16384.0
    accel_z = read_raw_data(i2c, ACCEL_XOUT_H + 4) / 16384.0
    gyro_x = read_raw_data(i2c, GYRO_XOUT_H) / 131.0
    gyro_y = read_raw_data(i2c, GYRO_XOUT_H + 2) / 131.0
    gyro_z = read_raw_data(i2c, GYRO_XOUT_H + 4) / 131.0

    return {
        'temp': temp,
        'accel': {
            'x': accel_x,
            'y': accel_y,
            'z': accel_z,
        },
        'gyro': {
            'x': gyro_x,
            'y': gyro_y,
            'z': gyro_z,
        }
    }
```

#ili9341.py

```
# Lightweight MicroPython ILI9341 driver
from micropython import const
import framebuf
import time

def color565(r, g, b):
    """Convert RGB888 to RGB565."""
    return ((r & 0xF8) << 8) | ((g & 0xFC) << 3) | (b >> 3)

class ILI9341(framebuf.FrameBuffer):
    def __init__(self, spi, cs, dc, rst, width=240, height=320):
        self.spi = spi
        self.cs = cs
```

```
        self.dc = dc
        self.rst = rst
        self.width = width
        self.height = height
        self.buffer = bytearray(width * height * 2)
        super().__init__(self.buffer, width, height, framebuf.RGB565)
        self.init_display()

    def _write_cmd(self, cmd):
        self.dc.value(0)
        self.cs.value(0)
        self.spi.write(bytearray([cmd]))
        self.cs.value(1)

    def _write_data(self, data):
        self.dc.value(1)
        self.cs.value(0)
        self.spi.write(data)
        self.cs.value(1)

    def init_display(self):
        # Hardware reset
        self.rst.value(1)
        time.sleep_ms(50)
        self.rst.value(0)
        time.sleep_ms(50)
        self.rst.value(1)
        time.sleep_ms(150)

        # Init sequence (simplified)
        self._write_cmd(0x01)  # Software reset
        time.sleep_ms(100)
        self._write_cmd(0x28)  # Display off
        self._write_cmd(0x36)
        self._write_data(bytearray([0x48]))
        self._write_cmd(0x3A)
        self._write_data(bytearray([0x55]))  # 16-bit color
        self._write_cmd(0x11)  # Sleep out
        time.sleep_ms(120)
        self._write_cmd(0x29)  # Display on
        time.sleep_ms(20)

    def fill(self, color):
        self._write_cmd(0x2C)
        data = bytearray(2)
        self.dc.value(1)
        self.cs.value(0)
        for y in range(self.height):
```

```
            for x in range(self.width):
                data[0] = color >> 8
                data[1] = color & 0xFF
                self.spi.write(data)
        self.cs.value(1)

    def text(self, string, x, y, color):
        super().text(string, x, y, color)
        self.show()

    def show(self):
        # Push buffer to display
        self._write_cmd(0x2C)
        self.dc.value(1)
        self.cs.value(0)
        self.spi.write(self.buffer)
        self.cs.value(1)
```

Experiment 18: Interface servo motor with Pico W

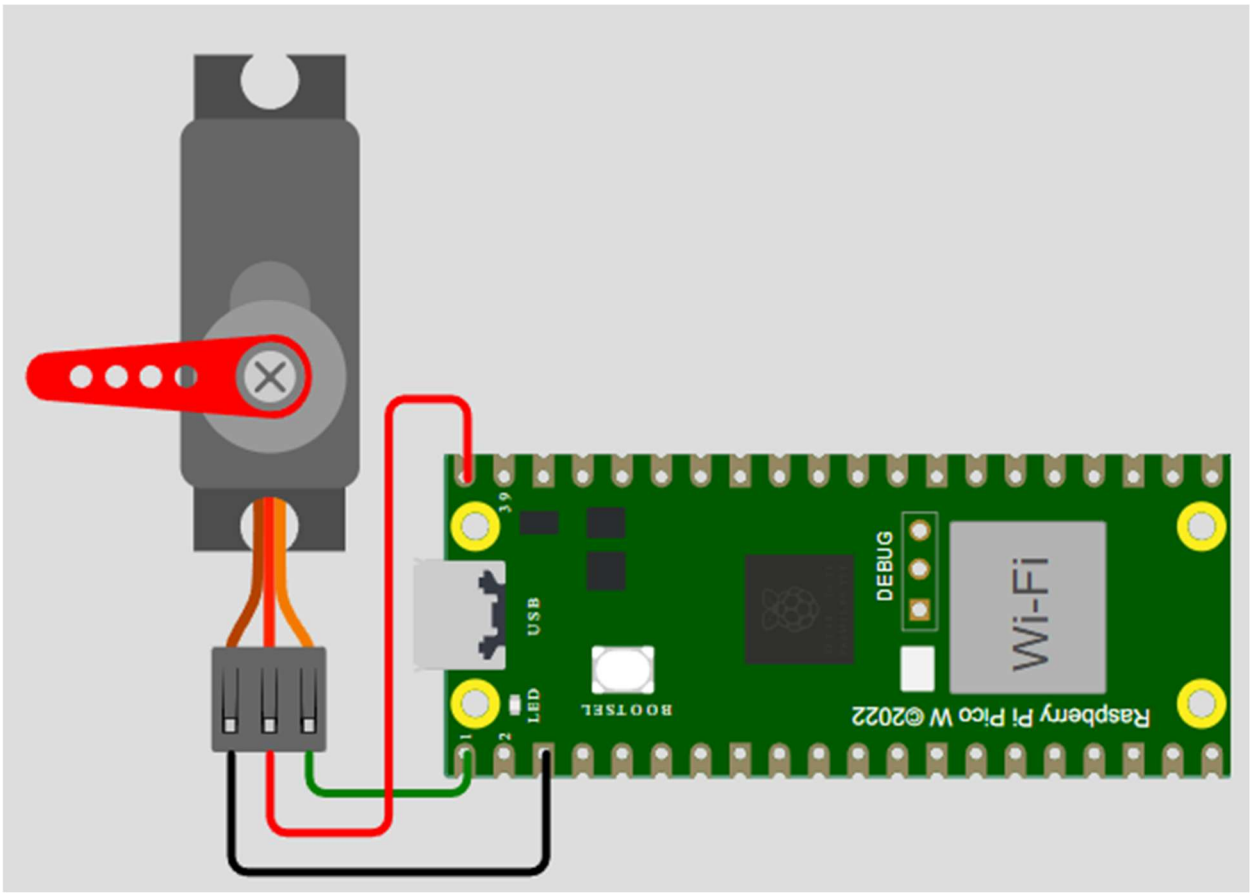


Fig 13: Circuit diagram for 18

The PWM signal has a fixed frequency, often around 50 Hz, with a duty cycle ranging from 5% to 10% representing the minimum angle and 10% to 20% representing the maximum angle of the servo motor.

Usually, the S0009 or SG90 servo motors (capable of 180°), used in most microcontroller projects, have a pulse width range of around **550 to 2400 microseconds**:

- Minimum Pulse Width (0°): around 550 microseconds
- Maximum Pulse Width (180°): around 2400 microseconds

The duty cycle is the ratio of the pulse-width to the total period of the signal. We can use the following formula:

- Duty cycle (%) = (pulse-width/period)x100

The period is the inverse of the frequency. Our frequency is 50Hz, which corresponds to 1/50 = 0.02 seconds (20000 microseconds).

So, to put the motor in 0° position, we need a 2.75% duty cycle:

Duty cycle = (550/20000)x100 = 2.75%

With the Raspberry Pi Pico, 100% duty cycle value is represented by a digital value of 65535, and 0% is represented by 0. So, we can map the duty cycle values to the Raspberry Pi Pico 0-65535 range as follows:

- Mapped value = Duty cycle x (maximum value/100)

So, the 0° position, would correspond to: Mapped value = 2.75 x (65535/100) = 1802

Doing the same procedure for the other values, we get:

- Minimum Pulse Width (0°) corresponds to 1802
- Maximum Pulse Width (180°) corresponds to 7864

#Code

#main.py

```
from machine import Pin, PWM
from time import sleep

# Set up PWM Pin for servo control
servo = PWM(Pin(0))

#Set PWM frequency
frequency = 50
servo.freq (frequency)

while True:
    #Go through all angles
    for pos in range(1000,9000,50):
        servo.duty_u16(pos)
        sleep(0.01)
    for pos in range(9000,1000,-50):
        servo.duty_u16(pos)
        sleep(0.01)
    sleep(2)
'''
    #Servo at 0 degrees
    servo.duty_u16(1802)
    sleep(2)
    #Servo at 90 degrees
    servo.duty_u16(4800)
    sleep(2)
    #Servo at 180 degrees
    servo.duty_u16(7984)
    sleep(2)
'''
```

Experiment 19: Interface 2 servo motors with Pico W and control them with an analog joystick.

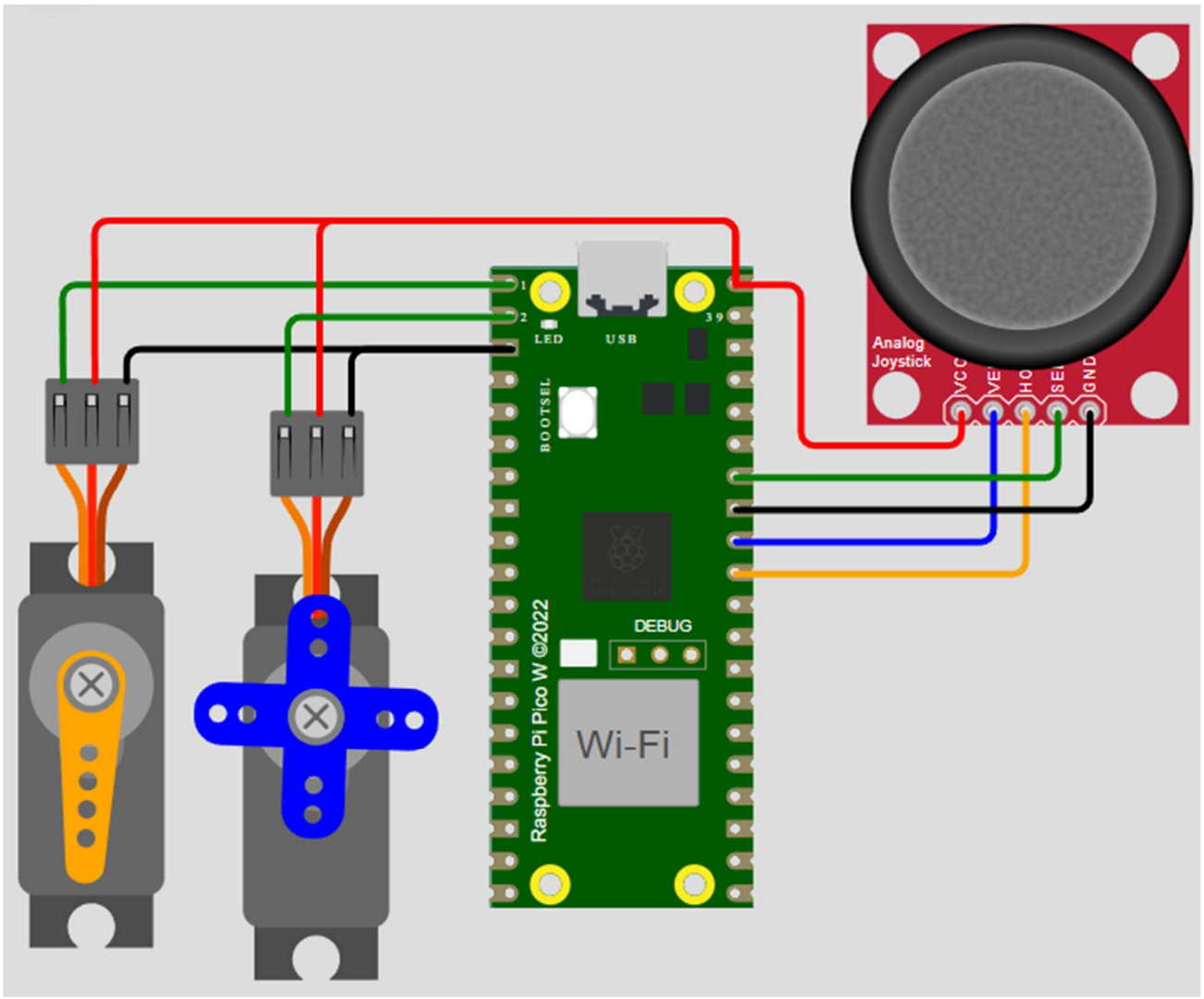


Fig 14: Circuit diagram for 19

#Code

#main.py

```
from machine import Pin, PWM, ADC
from time import sleep

class Servo:
    """ A simple class for controlling a 9g servo with the Raspberry Pi Pico.

    Attributes:

        minVal: An integer denoting the minimum duty value for the servo motor.
        maxVal: An integer denoting the maximum duty value for the servo motor.
```

```
    """

    def __init__(self, pin: int or Pin or PWM, minVal=1800, maxVal=8000):
        """ Creates a new Servo Object.
        args:
            pin (int or machine.Pin or machine.PWM): Either an integer denoting the number
of the GPIO pin or an already constructed Pin or PWM object that is connected to the
servo.
            minVal (int): Optional, denotes the minimum duty value to be used for this
servo.
            maxVal (int): Optional, denotes the maximum duty value to be used for this
servo.
        """
        if isinstance(pin, int):
            pin = Pin(pin, Pin.OUT)
        if isinstance(pin, Pin):
            self.__pwm = PWM(pin)
        if isinstance(pin, PWM):
            self.__pwm = pin
        self.__pwm.freq(50)
        self.minVal = minVal
        self.maxVal = maxVal

    def deinit(self):
        """ Deinitializes the underlying PWM object.
        """
        self.__pwm.deinit()

    def goto(self, value: int):
        """ Moves the servo to the specified position.
        args:
            value (int): The position to move to, represented by a value from 0 to 1024
(inclusive).
        """
        if value < 0:
            value = 0
        if value > 1024:
            value = 1024
        delta = self.maxVal-self.minVal
        target = int(self.minVal + ((value / 1024) * delta))
        self.__pwm.duty_u16(target)

    def middle(self):
        """ Moves the servo to the middle.
        """
        self.goto(512)

    def free(self):
```

```
        """ Allows the servo to be moved freely.
        """
        self.__pwm.duty_u16(0)
#End of class definition

s1 = Servo(0)  # Connected to GP0 and HORZ control
s2 = Servo(1)  # Connected to GP1 and VERT control
VRX = ADC(0)   # ADC0 multiplexing pin is GP26
VRY = ADC(1)   # ADC1 multiplexing pin is GP27
SW = Pin(28, Pin.IN, Pin.PULL_UP)  # ADC2 multiplexing pin is GP28

def Map(x, in_min, in_max, out_min, out_max):
    return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min

def servo_Angle(angle1,angle2):
    s1.goto(round(Map(angle1,0,180,0,1024)))
    s2.goto(round(Map(angle2,0,180,0,1024)))

def direction():
    global i
    i = 0
    adc_X=round(Map(VRX.read_u16(),0,65535,0,255))
    adc_Y=round(Map(VRY.read_u16(),0,65535,0,255))
    Switch = SW.value()
    if  adc_X <= 30:
        i = 1        # Define up direction
    elif adc_X >= 255:
        i = 2        # Define down direction
    elif adc_Y >= 255:
        i = 3        # Define left direction
    elif adc_Y <= 30:
        i = 4        # Define right direction
    elif  Switch == 0: #and adc_Y ==128:
        i = 5        # Define Button pressed
    elif adc_X - 125 < 15 and adc_X - 125 > -15 and adc_Y -125 < 15 and adc_Y -125 > -15
and Switch == 1:
        i = 0        # Define home location

def loop():
    num1 = 90
    num2 = 90
    while True:
        direction()   # Call the direction judgment function
        servo_Angle(num1,num2)
        sleep(0.01)
        if i == 1:
            num1 = num1 - 1
            if num1 < 0:
```

```
                num1 = 0
        if i == 2:
            num1 = num1 + 1
            if num1 > 180:
                num1 = 180
        if i == 3:
            num2 = num2 - 1
            if num2 < 0:
                num2 = 0
        if i == 4:
            num2 = num2 + 1
            if num2 > 180:
                num2 = 180
        if i==5:
            num1 = 90
            num2 = 90

if __name__ == '__main__':
    loop()   # call the loop function
```

Experiment 20: Interface a dc motor with Pico W

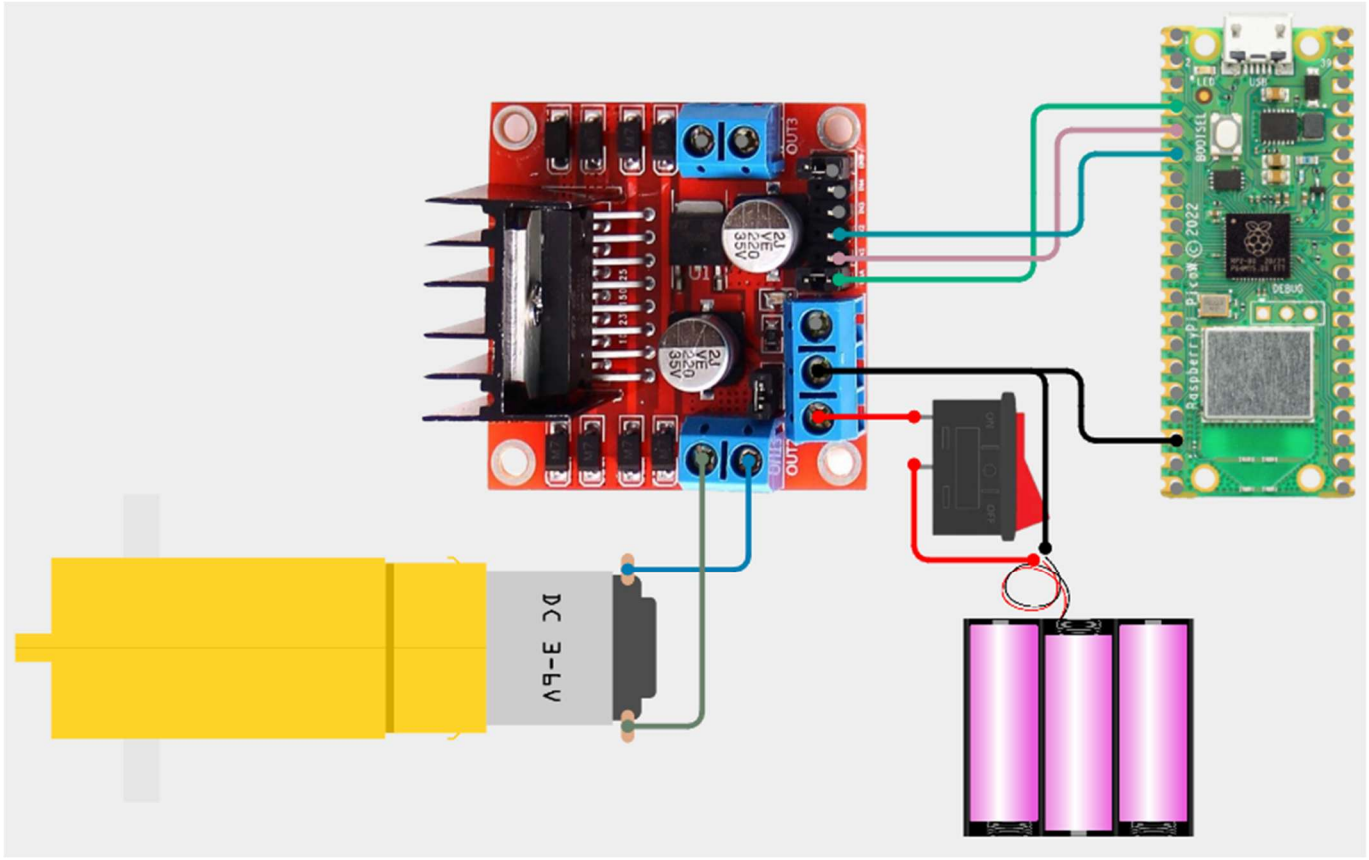


Fig 15: Circuit diagram for 20

SIGNAL ON THE ENABLE PIN	MOTOR STATE
HIGH	Motor enabled
LOW	Motor not enabled
PWM	Motor enabled: speed proportional to the duty cycle

For Motor A, the logic table is as shown:

Direction	Input 1	Input 2	Enable 1
Forward	0	1	1
Backwards	1	0	1
Stop	0	0	0

Table for connections between L298N and Pico W

L298N Motor Driver	Input 1	Input 2	Input 3	GND
Raspberry Pi Pico	GPIO 3	GPIO 4	GPIO 2	GND

#Code

#main.py

```
from machine import Pin, PWM
from time import sleep

class DCMotor:
    def __init__(self, pin1, pin2, enable_pin, min_duty=15000, max_duty=65535):
        self.pin1 = pin1
        self.pin2 = pin2
        self.enable_pin = enable_pin
        self.min_duty = min_duty
        self.max_duty = max_duty

    def forward(self, speed):
        self.speed = speed
        self.enable_pin.duty_u16(self.duty_cycle(self.speed))
        self.pin1.value(1)
```

```
        self.pin2.value(0)

    def backwards(self, speed):
        self.speed = speed
        self.enable_pin.duty_u16(self.duty_cycle(self.speed))
        self.pin1.value(0)
        self.pin2.value(1)

    def stop(self):
        self.enable_pin.duty_u16(0)
        self.pin1.value(0)
        self.pin2.value(0)

    def duty_cycle(self, speed):
        if speed <= 0 or speed > 100:
            duty_cycle = 0
        else:
            duty_cycle = int(self.min_duty + (self.max_duty - self.min_duty) * (speed /
100))
        return duty_cycle


frequency = 1000
pin1 = Pin(3, Pin.OUT)
pin2 = Pin(4, Pin.OUT)
enable = PWM(Pin(2), frequency)

dc_motor = DCMotor(pin1, pin2, enable)

# Set min duty cycle (15000) and max duty cycle (65535)
#dc_motor = DCMotor(pin1, pin2, enable, 15000, 65535)


print('Forward with speed: 50%')
dc_motor.forward(50)
sleep(5)
dc_motor.stop()
sleep(5)
print('Backwards with speed: 100%')
dc_motor.backwards(100)
sleep(5)
print('Forward with speed: 5%')
dc_motor.forward(5)
sleep(5)
dc_motor.stop()
```

Experiment 21: Interface 2 dc motors with Pico W

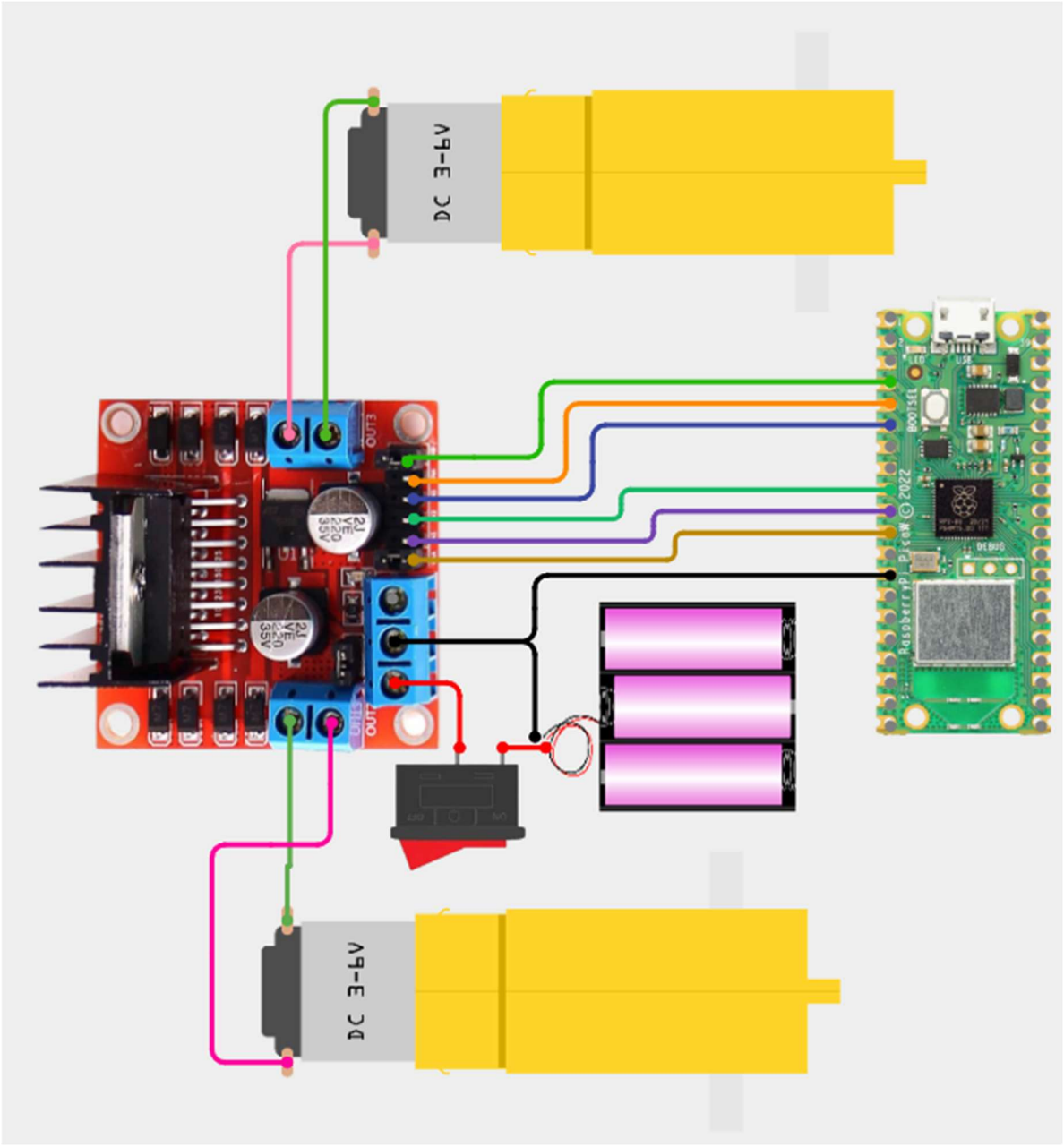


Fig 16: Circuit diagram for 21

#Code

#main.py

```
from machine import Pin, PWM
from time import sleep

class DCMotor:
    def __init__(self, pin1, pin2, enable_pin, min_duty=15000, max_duty=65535):
        self.pin1 = pin1
        self.pin2 = pin2
        self.enable_pin = enable_pin
        self.min_duty = min_duty
        self.max_duty = max_duty

    def forward(self, speed):
        self.speed = speed
        self.enable_pin.duty_u16(self.duty_cycle(self.speed))
        self.pin1.value(1)
        self.pin2.value(0)

    def backwards(self, speed):
        self.speed = speed
        self.enable_pin.duty_u16(self.duty_cycle(self.speed))
        self.pin1.value(0)
        self.pin2.value(1)

    def stop(self):
        self.enable_pin.duty_u16(0)
        self.pin1.value(0)
        self.pin2.value(0)

    def duty_cycle(self, speed):
        if speed <= 0 or speed > 100:
            duty_cycle = 0
        else:
            duty_cycle = int(self.min_duty + (self.max_duty - self.min_duty) * (speed /
100))
        return duty_cycle


frequency = 1000

#define inputs and outputs
ENA = PWM(Pin(8), frequency)
IN1 = Pin(7, Pin.OUT)
IN2 = Pin(6, Pin.OUT)
#ENA.freq(1000)
```

```
ENB = PWM(Pin(2), frequency)
IN3 = Pin(3, Pin.OUT)
IN4 = Pin(4, Pin.OUT)
#ENB.freq(1000)

dc_motor1 = DCMotor(IN1, IN2, ENA)
dc_motor2 = DCMotor(IN3, IN4, ENB)

while True:
    #Forward driving
    dc_motor1.forward(5)
    dc_motor2.forward(5)
    sleep(3)
    dc_motor1.stop()
    dc_motor2.stop()
    sleep(1)
    #Reversing
    dc_motor1.reverse(5)
    dc_motor2.reverse(5)
    sleep(3)
    dc_motor1.stop()
    dc_motor2.stop()
    sleep(1)
    #turn 1 side
    dc_motor1.forward(10)
    dc_motor2.reverse(5)
    sleep(2)
    dc_motor1.stop()
    dc_motor2.stop()
    sleep(1)
    #turn other side
    dc_motor1.reverse(5)
    dc_motor2.forward(10)
    sleep(2)
    dc_motor1.stop()
    dc_motor2.stop()
    sleep(1)
```

Experiment 22: Interface a stepper motor with Pico W.

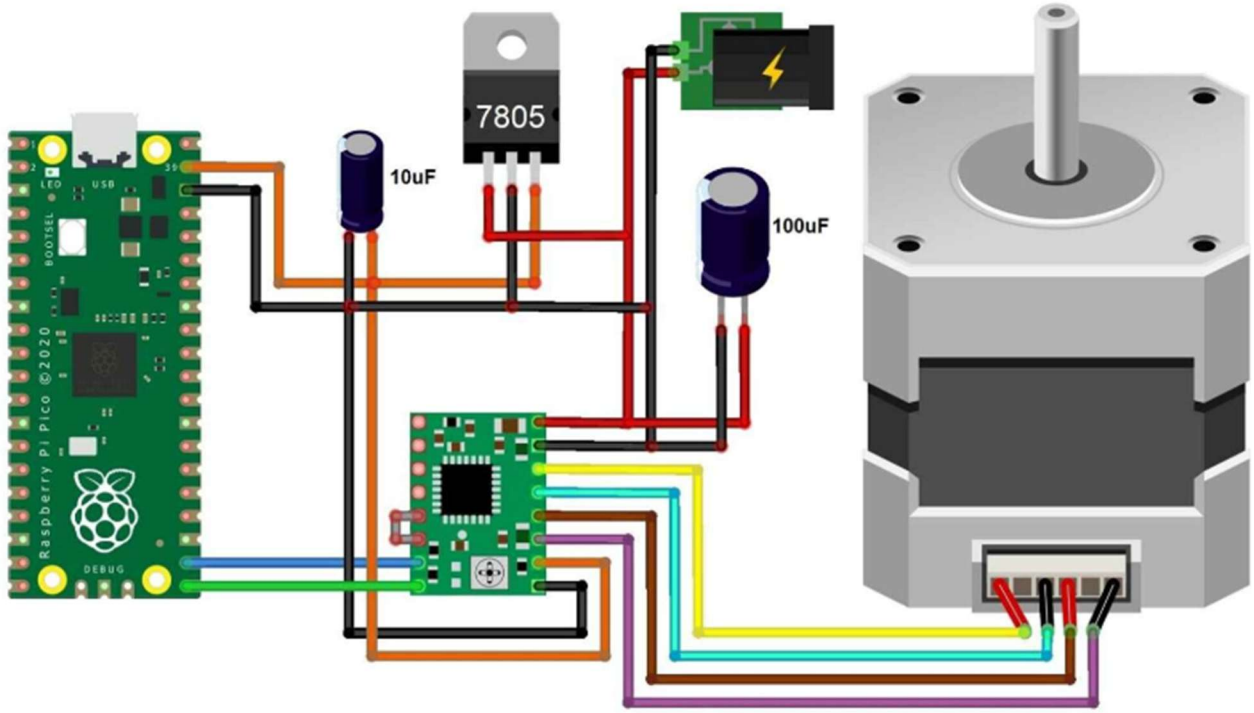


Fig 17: Circuit diagram for 12

#Code

#main.py

```
from machine import Pin, Timer
import utime

dir_pin = Pin(16, Pin.OUT)
step_pin = Pin(17, Pin.OUT)
steps_per_revolution = 200

# Initialize timer
tim = Timer()

def step(t):
    global step_pin
    step_pin.value(not step_pin.value())

def rotate_motor(delay):
    # Set up timer for stepping
    tim.init(freq=1000000//delay, mode=Timer.PERIODIC, callback=step)

def loop():
    while True:
```

```
        # Set  motor direction clockwise
        dir_pin.value(1)

        # Spin motor slowly
        rotate_motor(2000)
        utime.sleep_ms(steps_per_revolution)
        tim.deinit()  # stop the timer
        utime.sleep(1)

        # Set motor direction counterclockwise
        dir_pin.value(0)

        # Spin motor quickly
        rotate_motor(1000)
        utime.sleep_ms(steps_per_revolution)
        tim.deinit()  # stop the timer
        utime.sleep(1)

if __name__ == '__main__':
    loop()
```